Министерство науки и высшего образования Российской Федерации
Московский авиационный институт (национальный исследовательский университет)

Лабораторная работа №3

по дисциплине «Операционные системы»

Тема: управление потоками в ОС и синхронизация между потоками

Выполнил: Пьянзин М. А.
Группа: М8О-201БВ-25
Преподаватель: Е. С. Миронов
Вариант: 15

Москва, 2026

Постановка задачи

Цель работы: приобрести практические навыки управления потоками в операционной системе и обеспечения синхронизации между потоками.

Задание: составить программу на языке Си, обрабатывающую данные в многопоточном режиме с использованием стандартных средств создания потоков операционной системы. Максимальное число потоков, работающих в один момент времени, задаётся ключом запуска. Необходимо уметь продемонстрировать число потоков программы стандартными средствами ОС, а в отчёте исследовать зависимость ускорения и эффективности от входных данных и числа потоков и объяснить результаты.

Вариант: есть колода из 52 карт, нужно экспериментально (методом Монте-Карло) оценить вероятность того, что сверху лежат две одинаковые карты. Число раундов задаётся ключом запуска.

Теоретическое значение вероятности: $3/51 \approx 0{,}0588$. Оно используется для проверки корректности программы.

Описание метода и реализации

Один раунд: колода из 52 карт тасуется алгоритмом Фишера-Йетса, затем сравниваются номиналы (остаток от деления на 13) двух верхних карт. Если они равны, раунд считается успешным. Оценка вероятности равна числу успешных раундов, делённому на общее число раундов. Погрешность метода убывает пропорционально $1/\sqrt{N}$, где N число раундов.

Программа запускается как ./main <rounds> <threads> [seed], где rounds число раундов, threads число потоков, seed необязательное начальное значение генератора (по умолчанию берётся текущее время). Аргументы разбираются через strtoull и strtoul с проверкой ошибок.

Потоки создаются средствами POSIX Threads (pthread). Раунды делятся между потоками поровну, остаток раздаётся по одному лишнему раунду первым потокам, поэтому сумма раундов всегда равна rounds. Каждому потоку передаётся структура с его начальным значением генератора (seed + номер потока), числом раундов и полем для результата. Поток выполняет функцию carlo_alg и записывает число успехов в свою структуру. Главный поток создаёт все потоки через pthread_create, затем ждёт их через pthread_join и суммирует результаты. Если число потоков больше числа раундов, оно уменьшается до числа раундов.

Синхронизация. У каждого потока свои локальные данные: колода, счётчик успехов и состояние генератора rand_r, а результат записывается в отдельный элемент массива. Общих изменяемых данных во время счёта нет, поэтому мьютексы и семафоры не требуются, а гонки данных исключены. Корректность чтения результатов в главном потоке обеспечивает pthread_join: после его возврата поток гарантированно завершил запись. Вместо rand() использован rand_r, потому что у rand() общее скрытое состояние, и потоки были бы вынуждены ждать друг друга на его внутренней блокировке, что убило бы параллельность.

Исходный код

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <pthread.h>
#include <unistd.h>
#include "carlo.h"


typedef struct {
    unsigned int seed;
    unsigned long long rounds;
    unsigned long long success;
} task;

void *worker(void *arg){
    task *t = (task *)arg;
    unsigned long long s = carlo_alg(t->seed, t->rounds);
    t->success = s;
    return NULL;
}

int main(int argc, char *argv[]){
    if (argc < 3){
        fprintf(stderr, "Usage: %s <rounds> <threads> [seed]\n", argv[0]);
        return 1;
    }

    char *endp;
    unsigned long long rounds;
    unsigned int max_threads, seed;

    rounds = strtoull(argv[1], &endp, 10);
    if (*endp != '\0' || rounds == 0){
        fprintf(stderr, "Bad rounds: %s\n", argv[1]);
        return 1;
    }

    max_threads = (unsigned int)strtoul(argv[2], &endp, 10);
    if (*endp != '\0' || max_threads == 0){
        fprintf(stderr, "Bad threads: %s\n", argv[2]);
        return 1;
    }

    if (argc >= 4){
        seed = (unsigned int)strtoul(argv[3], &endp, 10);
        if (*endp != '\0'){
            fprintf(stderr, "Bad seed: %s\n", argv[3]);
            return 1;
```

```
        }
    } else {
        seed = (unsigned int)time(NULL);
    }
    if (max_threads > rounds) {
        max_threads = (unsigned int)rounds;
    }
    pthread_t threads[max_threads];
    task tasks[max_threads];

    for (unsigned int i = 0; i < max_threads; i++){
        tasks[i].rounds = rounds / max_threads;
        if (i < rounds % max_threads){
            tasks[i].rounds++;
        }
        tasks[i].seed = seed + i;
        tasks[i].success = 0;
    }

    struct timespec start, end;
    unsigned int created = 0;
    int th = 0;

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (unsigned int i = 0; i < max_threads; i++){
        th = pthread_create(&threads[i], NULL, worker, &tasks[i]);
            if (th != 0){
                fprintf(stderr, "pthread_create failed: %s\n", strerror(th));
                break;
            }
        created++;
    }

    for (unsigned int i = 0; i < created; i++){
        pthread_join(threads[i], NULL);
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    if (th != 0){
        return 1;
    }

    unsigned long long total = 0;
    for (unsigned int i = 0; i < max_threads; i++){
        total += tasks[i].success;
    }
    double prob = (double)total / rounds;

    double time_taken = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) /
1e9;

    printf("Probability: %f\nTime: %.3f\n", prob, time_taken);
    return 0;
}
```

carlo.c

```
#include <stdlib.h>
#include "carlo.h"

unsigned long long carlo_alg(unsigned int seed, unsigned long long rounds){
    int cards[52];
    unsigned long long success = 0;
    unsigned int state = seed;
    for(unsigned long long j =0; j < rounds;j++){
        for(int i =0; i <52; i++){
            cards[i]=i;
        }
```

```
        for(int k = 51; k >=0; k--){
            int temp = rand_r(&state) % (k + 1);
            int temp_card = cards[temp];
            cards[temp] = cards[k];
            cards[k] = temp_card;
        }
        if (cards[0]%13 == cards[1]%13){
            success++;
        }
    }
    return success;
}
```

carlo.h

```
#include <stdlib.h>

unsigned long long carlo_alg(unsigned int seed, unsigned long long rounds);
```

Демонстрация числа потоков

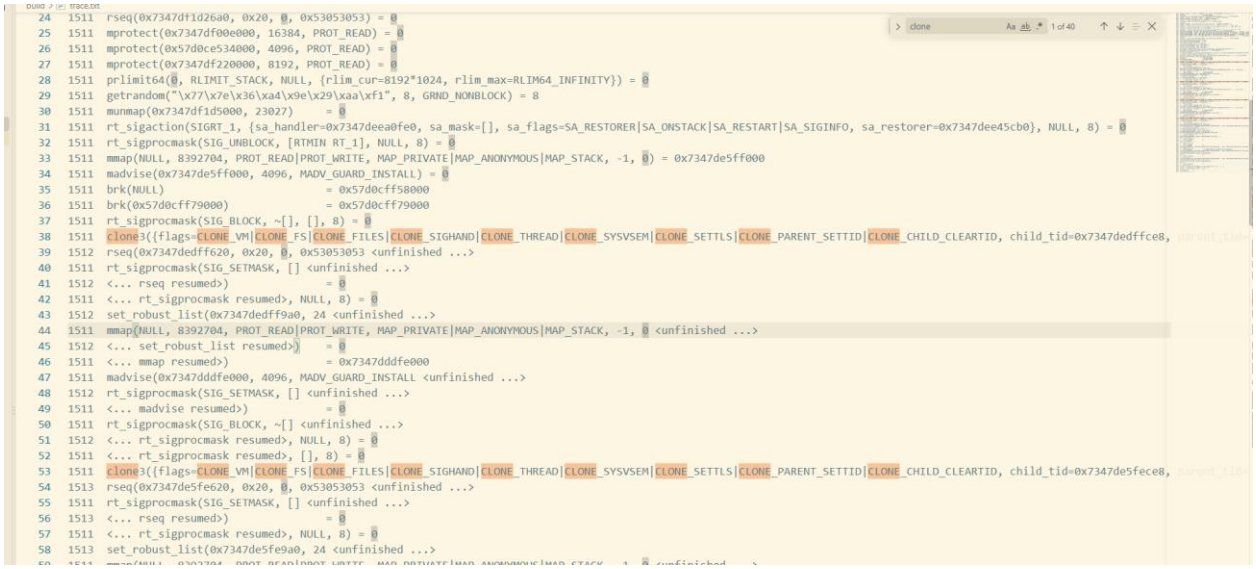
Для демонстрации программа запускалась с большим числом раундов, чтобы она работала десятки секунд: ./main 1000000000 4 42. Во втором терминале выполнялась команда top -H -p $(pgrep -n main). Ключ -H выводит каждый поток отдельной строкой.

Результат: пять потоков. Четыре рабочих потока в состоянии R (выполняются) загружают процессор примерно на 99,9% каждый, главный поток в состоянии S (спит) и не потребляет процессор, так как ожидает завершения рабочих в pthread_join. Размеры памяти (VIRT, RES, SHR) у всех потоков одинаковы, потому что потоки одного процесса используют общее адресное пространство.

```
pyanz@dropboy27:~/Code/OSIlab3$ top -H -p $(pgrep -n main)
Threads:   5 total,   4 running,   1 sleeping,   0 stopped,   0 zombie
%Cpu(s): 33.1 us,  0.0 sy,  0.0 ni, 66.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7904.8 total,   6181.6 free,   1476.2 used,    399.7 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   6428.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  17400 pyanz     20   0   35556   1916   1784 R  90.9   0.0   0:09.48 main
  17401 pyanz     20   0   35556   1916   1784 R  90.9   0.0   0:09.48 main
  17402 pyanz     20   0   35556   1916   1784 R  90.9   0.0   0:09.48 main
  17403 pyanz     20   0   35556   1916   1784 R  90.9   0.0   0:09.48 main
  17399 pyanz     20   0   35556   1916   1784 S   0.0   0.0   0:00.00 main
```

Создание потоков на уровне системных вызовов проверено через strace -f -o trace.txt ./main 1000000 4 42. В трассе есть четыре вызова clone3 с флагами CLONE_VM и CLONE_THREAD, которые вернули идентификаторы потоков. Флаг CLONE_VM означает, что новая задача использует то же адресное пространство, а CLONE_THREAD помещает её в ту же группу потоков, то есть у неё общий PID. Перед каждым clone3 библиотека выделяет через mmap отдельную область под стек потока. Главный поток засыпает в вызовах futex при ожидании в pthread_join, а завершение каждого рабочего потока видно как exit(0).



Исследование ускорения и эффективности

Время измерялось через clock_gettime(CLOCK_MONOTONIC) от первого pthread_create до последнего pthread_join.

```
pyanz@dropboy27:~/Code/OSIlab3/build$ ./main 1000000000 4 42
Probability: 0.058793
Time: 70.862
pyanz@dropboy27:~/Code/OSIlab3/build$ ./main 100000 1 42
Probability: 0.058590
Time: 0.028
pyanz@dropboy27:~/Code/OSIlab3/build$ ./main 100000 2 42
Probability: 0.058790
Time: 0.014
pyanz@dropboy27:~/Code/OSIlab3/build$ ./main 100000 4 42
Probability: 0.059210
Time: 0.008
pyanz@dropboy27:~/Code/OSIlab3/build$ ./main 100000 16 42
Probability: 0.058560
Time: 0.007
```

Ускорение близко к числу потоков, потому что раунды независимы, общих изменяемых данных нет и во время счёта нет блокировок и системных вызовов.

Выводы

В ходе выполнения лабораторной работы я получил практические навыки разработки многопоточных приложений на языке Си с использованием библиотеки POSIX Threads (pthread). Я научился эффективно распределять вычислительную нагрузку между независимыми потоками, организовывать корректный сбор результатов вычислений с помощью функции pthread_join, а также избегать состояний гонки (race conditions) без

использования тяжеловесных блокировок за счет грамотного проектирования архитектуры программы и изоляции данных внутри каждого потока.

Я освоил системные средства операционной системы Linux для мониторинга и отладки многопоточных процессов. Анализ трассировки системных вызовов с помощью strace помог глубже понять внутренние механизмы ОС, в частности, особенности создания потоков через системный вызов clone3 с флагами CLONE_VM и CLONE_THREAD, обеспечивающими работу в общем адресном пространстве.

Исследование эффективности показало, что для вычислительных задач с независимыми итерациями многопоточность обеспечивает ускорение. Это достигается благодаря полному отсутствию межпоточных блокировок и минимизации накладных расходов на системные вызовы во время параллельного счета, что подтверждает корректность выбранного подхода к оптимизации программы.